СЕМИНАР 7 · ВВЕДЕНИЕ В БАЗЫ ДАННЫХ

Подзапросы и общие табличные выражения

Что такое подзапрос

Подзапрос — запрос, который помещается внутрь другого SQL-запроса и чей результат используется основным (внешним) запросом для фильтрации данных, определения условий или в качестве источника данных.

Подзапросы позволяют:

- фильтровать строки по значениям из других таблиц
- сравнивать строки с агрегатами (среднее, максимум и т. п.)
- проверять существование или отсутствие связанных записей
- строить временные наборы данных для последующей фильтрации

Некоррелированные и коррелированные подзапросы

НЕКОРРЕЛИРОВАННЫЕ

- не используют столбцы внешнего запроса
- выполняются логически один раз, результат подставляется во внешний запрос

КОРРЕЛИРОВАННЫЕ

- ссылаются на столбцы внешнего запроса — выполняются для каждой строки внешнего набора
- связь — внешняя ссылка: ci.country_code = c.code

Страны с населением выше среднего

```
SELECT name
FROM w_country
WHERE population > (
    SELECT AVG(population)
    FROM w_country
);
```

Население страны выше среднего по её городам

```
SELECT c.name
FROM w_country c
WHERE c.population > (
    SELECT COALESCE(AVG(ci.population), 0)
    FROM w_city ci
    WHERE ci.country_code = c.code
);
```

Скалярные подзапросы, списки, табличные подзапросы

Вид	Что возвращает	Где используется
Скалярный	ровно 1 строку и 1 столбец — одно значение	как обычное выражение
Список	1 столбец, N строк	с IN, ANY / SOME, ALL
Табличный	много столбцов, много строк	в FROM — как временная таблица

Скалярный подзапрос вернул 0 строк → результат NULL.

Вернул больше 1 строки → ошибка.

Чтобы гарантированно получить одну строку, можно использовать LIMIT 1.

Где можно писать подзапросы

Предложение	Какие подзапросы	Корреляция
`SELECT`	скалярные — метрики и флаги в столбцах	коррелированные и некоррелированные
`FROM`	табличные, CTE (WITH), реже — скалярный	только некоррелированные
`WHERE`	скалярные; списки — IN, ANY, ALL	коррелированные и некоррелированные
`GROUP BY`	скалярные — очень редко	коррелированные и некоррелированные
`HAVING`	скалярные; списки — ANY, ALL	коррелированные и некоррелированные
`ORDER BY`	скалярные — сортировка по метрике, редко	коррелированные и некоррелированные
`LIMIT`	скалярные — динамический лимит, редко	только некоррелированные

Скалярный подзапрос можно писать везде, где ожидается выражение.

Скалярный подзапрос в WHERE

СИНТАКСИС

```
SELECT столбец1, ...
FROM таблица
WHERE столбец1 = (подзапрос)
```

ПРИМЕР

Найти страны с площадью, превышающей 80% от площади самой большой страны мира.

```
SELECT region,
       name,
       population
FROM w_country
WHERE surface_area > 0.8 * (
    SELECT MAX(surface_area)
    FROM w_country
);
```

- операторы: =, <>, <, <=, >, >=, BETWEEN, IS DISTINCT FROM, а также выражения в CASE, COALESCE
- некоррелированный вычисляется один раз — во все строки подставляется одно значение
- подзапрос вернул NULL → сравнение даёт UNKNOWN, строка отфильтруется; при необходимости — COALESCE

Коррелированный подзапрос в WHERE

- можно учесть значения из внешнего запроса: значение подзапроса своё для каждой строки
- вычисления выполняются несколько раз, в худшем случае — для каждой строки внешнего запроса

ПРИМЕР

Заказы, сумма которых выше среднего по этому же клиенту.

```
SELECT o.order_id,
       o.customer_id,
       o.total_amount
FROM shop_orders o
WHERE o.total_amount > (
    SELECT AVG(o2.total_amount)
    FROM shop_orders o2
    WHERE o2.customer_id = o.customer_id
);
```

Результат подзапроса для каждой строки

Строка внешнего запроса	Значение подзапроса
строка 1	5
строка 2	7
…	…
строка N	4

Скалярный подзапрос в HAVING

HAVING применяется после GROUP BY: сначала считаются агрегаты по группам, затем каждая группа сравнивается со значением подзапроса.

Некоррелированный: стран больше среднего

```
SELECT continent,
       COUNT(*) AS country_count
FROM w_country
GROUP BY continent
HAVING COUNT(*) > (
    SELECT AVG(country_count)
    FROM (SELECT continent,
                 COUNT(*) AS country_count
          FROM w_country
          GROUP BY continent) t
);
```

Коррелированный: больше среднего по стране

```
SELECT o.country_code,
       o.customer_id,
       COUNT(*) AS order_cnt
FROM shop_orders o
GROUP BY o.country_code, o.customer_id
HAVING COUNT(*) > (
    SELECT AVG(sub.order_cnt)
    FROM (SELECT customer_id,
                 COUNT(*) AS order_cnt
          FROM shop_orders s
          WHERE s.country_code = o.country_code
          GROUP BY customer_id) sub
);
```

IN / NOT IN: фильтрация списком

СИНТАКСИС

```
expr [NOT] IN (subquery)
```

- IN — TRUE, если expr равно какому-либо значению из подзапроса
- подзапрос возвращает список и может быть коррелированным и некоррелированным

Города в странах Европы

```
SELECT name
FROM w_city
WHERE country_code IN (
    SELECT code
    FROM w_country
    WHERE continent = 'Europe'
);
```

Страны без городов

```
SELECT name
FROM w_country c
WHERE c.code NOT IN (
    SELECT country_code
    FROM w_city
);
```

NOT IN опасен при NULL в подзапросе: одно NULL в списке делает результат UNKNOWN — строка исчезает. Безопаснее фильтровать NULL внутри подзапроса или применять NOT EXISTS.

ANY / SOME и ALL

Оператор	TRUE, если сравнение истинно	Подзапрос пуст	NULL в подзапросе
`expr <op> ANY (subquery)` `SOME` — синоним	хотя бы для одного значения	FALSE	влияет, только если нет ни одного TRUE
`expr <op> ALL (subquery)`	для всех значений	TRUE («вакуумная истина»)	нет FALSE, но есть UNKNOWN → UNKNOWN: исключай NULL в подзапросе

<op> — =, <>, <, <=, >, >=. Подзапрос может быть коррелированным и некоррелированным.

Больше хотя бы одной страны Европы

```
SELECT name
FROM w_country
WHERE population > ANY (
    SELECT population
    FROM w_country
    WHERE continent = 'Europe'
);
```

Больше всех стран Европы

```
SELECT name
FROM w_country
WHERE population > ALL (
    SELECT population
    FROM w_country
    WHERE continent = 'Europe'
      AND population IS NOT NULL
);
```

Списки в HAVING

IN, ANY, ALL можно применять к агрегатам; списки в HAVING используются довольно редко и специфично.

ПРИМЕР

Страны, у которых количество городов равно максимальному в стране на каком-нибудь континенте.

```
SELECT c.code, COUNT(*) AS city_cnt
FROM w_city ci
  JOIN w_country c
    ON c.code = ci.country_code
GROUP BY c.code
HAVING COUNT(*) IN (
    SELECT MAX(x.city_cnt) AS max_city_cnt     -- максимум по континенту
    FROM (SELECT country_code, COUNT(*) AS city_cnt
          FROM w_city
          GROUP BY country_code) x             -- городов в стране
      JOIN w_country wc
        ON wc.code = x.country_code
    GROUP BY wc.continent
);
```

EXISTS / NOT EXISTS: проверка существования строк

СИНТАКСИС

```
...
WHERE [NOT] EXISTS (
    SELECT <выражение>
    FROM table t
    WHERE <условие с внешними колонками>
)
```

- EXISTS возвращает TRUE, как только найдена первая подходящая строка
- содержимое SELECT внутри неважно — часто пишут SELECT 1
- дубликаты и NULL внутри подзапроса не влияют на результат
- NOT EXISTS — отрицание

Есть города с населением больше 1 млн

```
SELECT c.name
FROM w_country c
WHERE EXISTS (
    SELECT 1
    FROM w_city ci
    WHERE ci.country_code = c.code
      AND ci.population > 1000000
);
```

Страны без городов

```
SELECT c.name
FROM w_country c
WHERE NOT EXISTS (
    SELECT 1
    FROM w_city ci
    WHERE ci.country_code = c.code
);
```

Полусоединение (semi join)

Вернуть строки только из левой таблицы при наличии совпадений справа — без связывания строк: проверить факт наличия, не возвращая столбцы правой таблицы.

Способ	Запрос: страны с городами	Замечание

EXISTS — рекомендуется	`SELECT c.*`
	`FROM w_country c`
	`WHERE EXISTS (SELECT 1`
	`FROM w_city ci`
	`WHERE ci.country_code = c.code);`
		обычно быстрее и надёжнее

IN	`SELECT c.*`
	`FROM w_country c`
	`WHERE c.code IN (SELECT country_code`
	`FROM w_city);`
		дубли и NULL в подзапросе на результат не влияют

INNER JOIN + DISTINCT	`SELECT DISTINCT c.*`
	`FROM w_country c`
	`JOIN w_city ci`
	`ON ci.country_code = c.code;`
		без DISTINCT страна повторится по разу на каждый город

Антисоединение (anti join)

Вернуть строки из левой таблицы, для которых нет совпадений справа: найти «висячие» записи — страны без городов, заказы без платежей.

Способ	Запрос: страны без городов	Замечание

NOT EXISTS	`SELECT c.*`
	`FROM w_country c`
	`WHERE NOT EXISTS (SELECT 1 FROM w_city ci`
	`WHERE ci.country_code = c.code);`
		надёжный способ

NOT IN	`SELECT c.*`
	`FROM w_country c`
	`WHERE c.code NOT IN (SELECT country_code FROM w_city`
	`WHERE country_code IS NOT NULL);`
		только если исключены NULL

LEFT JOIN … IS NULL	`SELECT c.*`
	`FROM w_country c`
	`LEFT JOIN w_city ci`
	`ON ci.country_code = c.code`
	`WHERE ci.country_code IS NULL;`
		строка не нашла пару справа

При LEFT JOIN условия на правую таблицу — в ON: условие в WHERE превратит левое соединение во внутреннее.

Типовые ошибки

Ошибка	Плохо	Как надо

Скалярный подзапрос вернул больше 1 строки	`WHERE population = (`
	`  SELECT population FROM w_country`
	`  WHERE continent = 'Europe')`
		`WHERE population = (`
		`  SELECT MAX(population) ...)`
		`-- или IN (SELECT population ...)`

NOT IN и NULL	`WHERE c.code NOT IN (`
	`  SELECT country_code FROM w_city)`
		`WHERE NOT EXISTS (`
		`  SELECT 1 FROM w_city x`
		`  WHERE x.country_code = c.code)`

Несовместимые типы	`WHERE customer_id IN (`
	`  SELECT email_hash`
	`  FROM shop_customers)  -- int и text`
		`WHERE customer_id::text IN (`
		`  SELECT email_hash`
		`  FROM shop_customers)`

ALL / ANY без фильтра на NULL	`WHERE population > ALL (`
	`  SELECT population FROM w_country`
	`  WHERE continent = 'Europe')`
		`... WHERE continent = 'Europe'`
		`  AND population IS NOT NULL)`

Ожидание порядка внутри подзапроса	`SELECT *`
	`FROM (SELECT name FROM w_city`
	`  ORDER BY population DESC) t;`
		`ORDER BY без LIMIT в подзапросе`
		`не гарантирует порядок снаружи`

Общие табличные выражения: WITH

CTE (Common Table Expressions) — временные именованные результирующие наборы, доступные в рамках одного SQL-запроса. Помогают структурировать сложные запросы, повышают читаемость и позволяют писать рекурсивные вычисления.

СИНТАКСИС

```
WITH cte_name AS (
    SELECT ...
)
SELECT ...
FROM cte_name;
```

CTE логически работает как временное представление, которое существует только в пределах выполняемого запроса.

ПРИМЕР: выручка по товарам

```
WITH sales AS (
    SELECT product_id,
           SUM(amount) AS rev
    FROM shop_orders
    GROUP BY product_id
)
SELECT s.product_id,
       s.rev
FROM sales s;
```

Материализация CTE

До PostgreSQL 12: обязательная материализация

```
WITH big_cte AS (
    SELECT *
    FROM orders
    WHERE amount > 1000
)
SELECT *
FROM big_cte
WHERE customer_id = 10;
```

CTE выполняется целиком, результат сохраняется, условие customer_id = 10 применяется потом: оптимизационный барьер.

С PostgreSQL 12: встраивание (inline)

```
-- для планировщика тот же запрос
SELECT *
FROM orders
WHERE amount > 1000
  AND customer_id = 10;
```

Фильтры объединяются при построении плана, промежуточный набор не формируется.

```
WITH cte_name AS MATERIALIZED (...)       -- принудительно создать промежуточный результат
WITH cte_name AS NOT MATERIALIZED (...)   -- разрешить встроить CTE в основной запрос
```

По умолчанию PostgreSQL решает сам по оценке стоимости выполнения.

Несколько CTE и CTE для фильтрации

Несколько CTE — через запятую

```
WITH first_cte AS (
    SELECT id, name
    FROM employee
    WHERE head_id IS NULL
),
second_cte AS (
    SELECT id, name
    FROM employee
    WHERE head_id IS NOT NULL
)
SELECT * FROM first_cte
UNION ALL
SELECT * FROM second_cte;
```

Категории с продажами выше средней по категориям

```
WITH category_sales AS (
    SELECT p.category_id,
           SUM(o.total_amount) AS total_sales
    FROM shop_products p
      JOIN shop_orderitems oi
        ON p.product_id = oi.product_id
      JOIN shop_orders o
        ON oi.order_id = o.order_id
    GROUP BY p.category_id
)
SELECT category_id, total_sales
FROM category_sales
WHERE total_sales > (
    SELECT AVG(total_sales) FROM category_sales
);
```

Рекурсивные CTE: уровни подчинённости

```
WITH RECURSIVE hierarchy AS (
    SELECT id, name, head_id,
           1 AS level
    FROM employee
    WHERE head_id IS NULL          -- якорь
    UNION ALL
    SELECT e.id, e.name, e.head_id,
           h.level + 1
    FROM employee e
      JOIN hierarchy h
        ON e.head_id = h.id
    WHERE h.level < 3              -- глубина
)
SELECT *
FROM hierarchy
ORDER BY id;
```

- **якорный запрос** формирует начальный набор данных
- **рекурсивный запрос** ссылается на CTE и выполняется, пока не обработаны все уровни; части соединяются UNION ALL

employee

id	name	head_id
1	Елена	NULL
2	Кирилл	1
3	Андрей	2
4	Анастасия	1
5	Александр	4
6	Константин	4
7	Ольга	6
8	Мария	1
9	Владислав	7
10	Олег	4

Результат — 8 строк:

уровень 1 — Елена;

уровень 2 — Кирилл, Анастасия, Мария;

уровень 3 — Андрей, Александр, Константин, Олег.

Ольга и Владислав (уровни 4 и 5) отсечены условием h.level < 3.

Рекурсивные CTE: последовательность чисел

```
WITH RECURSIVE numbers(n) AS (
    SELECT 1
    UNION ALL
    SELECT n + 1
    FROM numbers
    WHERE n < 10
)
SELECT n FROM numbers;
```

Результат: 10 строк — числа от 1 до 10

numbers(n) задаёт имя столбца: без него рекурсивной части не на что сослаться.

КАК РАБОТАЕТ

1. Якорная часть создаёт первую строку: n = 1

2. Рекурсивная часть читает столбец n из текущего результата

3. Создаёт новую строку n = 2 и добавляет её в конец

4. Повторяет, пока выполняется n < 10

Рекурсия останавливается, когда рекурсивная часть вернёт 0 строк.

ИТОГИ

Подведём итоги

Что нужно	Как пишется
Сравнить строку или группу со значением по всей таблице	`WHERE / HAVING x > (SELECT AVG(x) FROM …)`
Значение, своё для каждой строки	`коррелированный подзапрос: … WHERE t2.key = t1.key`
Проверить, входит ли значение в список	`IN, ANY / SOME, ALL`
Проверить, есть ли пара	`EXISTS, IN`
Проверить, нет ли пары	`NOT EXISTS; NOT IN — только без NULL`
Именованный шаг запроса	`WITH имя AS (…)`
Иерархия, последовательность	`WITH RECURSIVE`